

Logbook

Advanced MicroController Q4

Contents

1. DMX 512	3
1.1. What I Did	3
1.2. Timing Calculations	3
1.3. Writing the PIO Code	3
1.4. C Code and Testing	3
2. Power	4
2.1. Measurements	4
2.2. Analysis	4
2.3. Sleep and Wake Mode	4
2.4. I2C Configuration	4
2.5. Controller Code (Master Pico)	5
2.6. Slave Pico Code	10
3. Multicore	13
3.1. Assignment Part 2: Multicore Communication with Strings	15
3.1.1. Changes Made: Strings Instead of Chars	15
3.1.2. Technical Note	16
4. Security & OTP	17
4.1. Step 1: Generate a Private Key	17
4.2. Step 2: Updating CMakeLists	17
4.3. Step 3: Checking the Build Files	17
4.4. Step 4: OTP Loading	17
5. OTA Updates	18
6. USB as Host	20
7. Bluetooth	21
7.1. Assignment BLE — Multiple Choice & Short Questions	21
7.2. Assignment: Classic Bluetooth SPP Server	22
7.2.1. What I Implemented	22
8. DMA	23
8.1. Assignment: DMA Demo	23

1. DMX 512

1.1. What I Did

Started by looking at the given `hello.pio` template. The break part was already there, holding the pin low for 124us (31 loops * 4us). My job was to add the MAB (mark after break) and the actual data transmission part.

1.2. Timing Calculations

The most important part of this is the timing. The Pico runs at 125MHz. We set the PIO clock div to 125.0 in the C code: `sm_config_set_clkdiv(&c, 125.0f);`

This is convenient because: $125\text{MHz} / 125 = 1\text{MHz}$, so 1 PIO cycle is exactly 1us. DMX runs at 250kbaud. $1 / 250000 = 4\text{us}$ per bit, so every bit needs to take exactly 4 cycles in PIO.

1.3. Writing the PIO Code

1. **MAB:** After the break, the line needs to go high for at least 8us. Set it high and looped for 12us total for safety.
2. **Data loop:**
 - First it pulls the amount of bytes from OSR (copied to Y register).
 - Inside `byte_loop`, it pulls the actual data byte.
 - **Start bit:** Line goes low for 4us (`set pins, 0 [3]`).
 - **Data bits:** A `bit_loop` shifts out 8 bits, 4us each (`out pins, 1 [3]`). It shifts right (LSB first) because of `sm_config_set_out_shift(&c, true, false, 8);`.
 - **Stop bits:** Line goes high for 8us (2 stop bits, `set pins, 1 [7]`).
 - Repeats for all bytes (`jmp y-- byte_loop`).

1.4. C Code and Testing

In `hello.c`, the CPU sends 512 (number of channels + start code) to tell PIO how many bytes to expect. Then it sends the start code (0x00) and 512 channels of data.

To test it, a heartbeat was added to the onboard LED. Using the YD-RP2040 board (cheap RP2040 breakout from AliExpress), the standard LED is on GPIO 25 (the blue one, not the RGB NeoPixel on GPIO 23). The LED turns on before feeding the PIO FIFO and off after. After flashing, the blue LED blinks rapidly — this proves the PIO state machine is processing data and not getting stuck waiting on the FIFO. Next step would be hooking it up to a MAX485 chip to drive a real light.

2. Power

2.1. Measurements

Mode	GPIO	Shunt Voltage	Current	Power
External LED	16	-0.050375 V	50 mA	0.165 W
Servo	17	-0.080375 V	51 mA	0.264 W
Sleep	19	-0.001480 V	0.56 mA	0.00008 W
Hanging	00	-0.003620 V	13.8 mA	0.002 W

2.2. Analysis

The servo consumed the most power at 0.264 W — expected, since it has a motor that needs to hold position. The external LED came in second at 0.165 W. Sleep mode dropped consumption to nearly nothing at 0.00008 W, which is exactly why dormant mode exists. The idle “hanging” state with no load sat at 0.002 W.

2.3. Sleep and Wake Mode

```
r
reading
Shunt Voltage: -0.001480 V
Current: -0.005646 A
Power: 0.000080 W
wake
  Wake mode (GPIO 19 LOW)
r
reading
Shunt Voltage: -0.003615 V
Current: -0.013809 A
Power: 0.000202 W
r
reading
Shunt Voltage: -0.003620 V
Current: -0.013809 A
```

Figure 1: Sleep and Wake Setup

2.4. I2C Configuration

- Port: I2C0
- INA238 Register: 0x04 (Shunt Voltage)

2.5. Controller Code (Master Pico)

```
#include <stdio.h>
#include <string.h>
#include <pico/stdlib.h>
#include <hardware/i2c.h>

#define I2C_PORT i2c0
#define I2C_SDA 8
#define I2C_SCL 9

#define INA238_ADRES 0x45
#define INA238_CALIBRATION_REG 0x02
#define SHUNT_VOLTAGE_REG 0x04
#define BUS_VOLTAGE_REG 0x05
#define POWER_REG 0x08
#define CURRENT_REG 0x07

#define MAX17048_ADDR 0x36
#define VCELL_REG 0x02
#define SOC_REG 0x04
#define MAX17048_MODE_REG 0x06
#define MAX17048_VERSION_REG 0x08

#define MAX17048_QUICKSTART_CMD 0x4000u
#define MAX17048_READY_MASK 0xFFFF0u
#define MAX17048_READY_VALUE 0x0010u

#define INA238_MAX_CURRENT_AMPS 10.0f
#define INA238_SHUNT_RESISTANCE_OHMS 0.01f

#define GPIO_EXTERNAL_LED 16
#define GPIO_SERVO 17
#define GPIO_INTERNAL_LED 18
#define GPIO_SLEEP_TRIGGER 19

// CURRENT_LSB = Max Current / 2^15 = 10A / 32768
#define CURRENT_LSB (INA238_MAX_CURRENT_AMPS / 32768.0f)

bool i2c_read_register16(uint8_t address, uint8_t reg, uint16_t *value);
bool i2c_write_register16(uint8_t address, uint8_t reg, uint16_t value);
void ina238_init(void);
bool max17048_init(void);
bool max17048_quick_start(void);
float ina238_getShuntVoltage();
float ina238_getCurrent();
float ina238_getPower();
bool max17048_getVoltage(float *voltage);
```

```
bool max17048_getSOC(float *soc);

static bool max17048_ready = false;

void set_mode_external() {
    gpio_put(GPIO_EXTERNAL_LED, 1);
    gpio_put(GPIO_SERVO, 0);
    gpio_put(GPIO_INTERNAL_LED, 0);
    gpio_put(GPIO_SLEEP_TRIGGER, 0);
    printf("\n External LED   (GPIO 16 pulse 10ms)\n");
    sleep_ms(10);
    gpio_put(GPIO_EXTERNAL_LED, 0);
}

void set_mode_internal() {
    gpio_put(GPIO_EXTERNAL_LED, 0);
    gpio_put(GPIO_SERVO, 0);
    gpio_put(GPIO_INTERNAL_LED, 1);
    gpio_put(GPIO_SLEEP_TRIGGER, 0);
    printf("\n Internal LED   (GPIO 18 pulse 10ms)\n");
    sleep_ms(10);
    gpio_put(GPIO_INTERNAL_LED, 0);
}

void set_mode_servo() {
    gpio_put(GPIO_EXTERNAL_LED, 0);
    gpio_put(GPIO_SERVO, 1);
    gpio_put(GPIO_INTERNAL_LED, 0);
    gpio_put(GPIO_SLEEP_TRIGGER, 0);
    printf("\n Servo mode   (GPIO 17 pulse 10ms)\n");
    sleep_ms(10);
    gpio_put(GPIO_SERVO, 0);
}

void set_mode_sleep() {
    gpio_put(GPIO_EXTERNAL_LED, 0);
    gpio_put(GPIO_SERVO, 0);
    gpio_put(GPIO_INTERNAL_LED, 0);
    gpio_put(GPIO_SLEEP_TRIGGER, 1);
    printf("\n  Sleep mode (GPIO 19 HIGH)\n");
}

void set_mode_wake() {
    gpio_put(GPIO_EXTERNAL_LED, 0);
    gpio_put(GPIO_SERVO, 0);
    gpio_put(GPIO_INTERNAL_LED, 0);
    gpio_put(GPIO_SLEEP_TRIGGER, 1);
}
```

```
sleep_ms(2);
gpio_put(GPIO_SLEEP_TRIGGER, 0);
sleep_ms(2);
printf("\n Wake mode (GPIO 19 LOW)\n");
}

int main() {
    stdio_init_all();
    i2c_init(I2C_PORT, 400*1000);
    gpio_set_function(I2C_SDA, GPIO_FUNC_I2C);
    gpio_set_function(I2C_SCL, GPIO_FUNC_I2C);
    gpio_pull_up(I2C_SDA);
    gpio_pull_up(I2C_SCL);

    gpio_init(GPIO_EXTERNAL_LED); gpio_set_dir(GPIO_EXTERNAL_LED, GPIO_OUT);
    gpio_put(GPIO_EXTERNAL_LED, 0);
    gpio_init(GPIO_SERVO);          gpio_set_dir(GPIO_SERVO, GPIO_OUT);
    gpio_put(GPIO_SERVO, 0);
    gpio_init(GPIO_INTERNAL_LED); gpio_set_dir(GPIO_INTERNAL_LED, GPIO_OUT);
    gpio_put(GPIO_INTERNAL_LED, 0);
    gpio_init(GPIO_SLEEP_TRIGGER); gpio_set_dir(GPIO_SLEEP_TRIGGER,
    GPIO_OUT); gpio_put(GPIO_SLEEP_TRIGGER, 0);

    ina238_init();
    max17048_ready = max17048_init();

    printf(" 'r'          - Read power and battery measurements\n");
    printf(" 'battquick'- Re-run MAX17048 quick-start\n");
    printf(" 'external' - External LED mode (GPIO 16)\n");
    printf(" 'internal' - Internal LED mode (GPIO 18)\n");
    printf(" 'servo'    - Servo mode (GPIO 17)\n");
    printf(" 'sleep'    - Sleep mode (GPIO 19)\n");
    printf(" 'wake'     - Wake slave (GPIO 19 LOW)\n");

    char command_buffer[20] = {0};
    int command_index = 0;

    while (true) {
        int c = getchar_timeout_us(0);
        if (c != PICO_ERROR_TIMEOUT) {
            if (c == '\n' || c == '\r') {
                command_buffer[command_index] = '\0';
                if (strcmp(command_buffer, "r") == 0) {
                    printf("\nreading \n");
                    printf("Shunt Voltage: %.6f V\n",
                    ina238_getShuntVoltage());
                    printf("Current: %.6f A\n",          ina238_getCurrent());
                }
            }
        }
    }
}
```

```
        printf("Power: %.6f W\n",          ina238_getPower());
        if (max17048_ready) {
            float v = 0, s = 0;
            printf("Battery Voltage: %s\n",
max17048_getVoltage(&v) ? "" : "unavailable");
            if (max17048_getVoltage(&v)) printf("%.3f V\n", v);
            if (max17048_getSOC(&s))      printf("Battery SOC: %.2f
%%\n", s);
            else                          printf("Battery SOC:
unavailable\n");
        } else printf("Battery Gauge: not ready\n");
    }
    else if (strcmp(command_buffer, "battquick") == 0) {
        printf(max17048_ready && max17048_quick_start()
? "\n MAX17048 quick-start issued\n"
: "\n MAX17048 quick-start failed\n");
        if (max17048_ready) sleep_ms(500);
    }
    else if (strcmp(command_buffer, "external") == 0)
set_mode_external();
    else if (strcmp(command_buffer, "internal") == 0)
set_mode_internal();
    else if (strcmp(command_buffer, "servo") == 0)
set_mode_servo();
    else if (strcmp(command_buffer, "sleep") == 0)
set_mode_sleep();
    else if (strcmp(command_buffer, "wake") == 0)
set_mode_wake();
        command_index = 0; command_buffer[0] = '\0';
    } else if (command_index < 19) {
        command_buffer[command_index++] = c;
        printf("%c", c);
    }
    }
    sleep_ms(10);
}

bool i2c_read_register16(uint8_t address, uint8_t reg, uint16_t *value) {
    uint8_t buf[2];
    if (i2c_write_blocking(I2C_PORT, address, &reg, 1, true) != 1) return
false;
    if (i2c_read_blocking(I2C_PORT, address, buf, 2, false) != 2) return
false;
    *value = ((uint16_t)buf[0] << 8) | buf[1];
    return true;
}
```

```
bool i2c_write_register16(uint8_t address, uint8_t reg, uint16_t value) {
    uint8_t buf[3] = {reg, (uint8_t)(value >> 8), (uint8_t)(value & 0xFF)};
    return i2c_write_blocking(I2C_PORT, address, buf, 3, false) == 3;
}

void ina238_init(void) {
    const float cal = 819.2e6f * CURRENT_LSB * INA238_SHUNT_RESISTANCE_OHMS;
    const uint16_t cal_reg = (uint16_t)cal;
    printf(i2c_write_register16(INA238_ADRES, INA238_CALIBRATION_REG, cal_reg)
        ? "INA238 calibrated (CAL=0x%04X)\n"
        : "INA238 calibration failed\n", cal_reg);
}

bool max17048_quick_start(void) {
    return i2c_write_register16(MAX17048_ADDR, MAX17048_MODE_REG,
MAX17048_QUICKSTART_CMD);
}

bool max17048_init(void) {
    uint16_t version = 0;
    if (!i2c_read_register16(MAX17048_ADDR, MAX17048_VERSION_REG, &version)) {
        printf("MAX17048 not detected\n"); return false;
    }
    if ((version & MAX17048_READY_MASK) != MAX17048_READY_VALUE) {
        printf("MAX17048 version check failed: 0x%04X\n", version); return
false;
    }
    printf("MAX17048 ready (version 0x%04X)\n", version);
    sleep_ms(1000);
    printf(max17048_quick_start()
        ? "MAX17048 quick-start issued at startup\n"
        : "MAX17048 quick-start skipped\n");
    if (max17048_quick_start()) sleep_ms(500);
    return true;
}

float ina238_getShuntVoltage() {
    uint16_t raw = 0;
    if (!i2c_read_register16(INA238_ADRES, SHUNT_VOLTAGE_REG, &raw)) return
0.0f;
    return (float)(int16_t)raw * 5.0f / 1000000.0f;
}

float ina238_getCurrent() {
    uint16_t raw = 0;
    if (!i2c_read_register16(INA238_ADRES, CURRENT_REG, &raw)) return 0.0f;
```

```
    return (float)(int16_t)raw * CURRENT_LSB;
}

float ina238_getPower() {
    uint16_t result = 0;
    if (!i2c_read_register16(INA238_ADRES, POWER_REG, &result)) return 0.0f;
    return 0.2f * CURRENT_LSB * (float)result;
}

bool max17048_getVoltage(float *voltage) {
    uint16_t result = 0;
    if (!i2c_read_register16(MAX17048_ADDR, VCELL_REG, &result)) return false;
    *voltage = (float)result * 5.12f / 65536.0f;
    return true;
}

bool max17048_getSOC(float *soc) {
    uint16_t result = 0;
    if (!i2c_read_register16(MAX17048_ADDR, SOC_REG, &result)) return false;
    *soc = (float)(result >> 8) + ((float)(result & 0x00FF) / 256.0f);
    return true;
}
```

2.6. Slave Pico Code

```
#include "pico/stdlib.h"
#include "hardware/pwm.h"
#include "hardware/irq.h"
#include "pico/sleep.h"
#include <stdio.h>

#ifdef CYW43_WL_GPIO_LED_PIN
#include "pico/cyw43_arch.h"
#endif

#define LED_DELAY_MS 250
#define EXT_LED_PIN 0
#define SERVO_PIN 2
#define BTN_EXT_LED 16
#define BTN_SERVO 17
#define BTN_INTERNAL 18
#define BTN_SLEEP 19
#define SERVO_ON_US 1000u
#define SERVO_OFF_US 1500u

static bool ext_led_state = false;
```

```
static bool servo_state    = false;
static bool internal_state = false;
static bool sleeping       = false;
static uint servo_slice;

int pico_led_init(void) {
#ifdef PICO_DEFAULT_LED_PIN
    gpio_init(PICO_DEFAULT_LED_PIN);
    gpio_set_dir(PICO_DEFAULT_LED_PIN, GPIO_OUT);
    return PICO_OK;
#elif defined(CYW43_WL_GPIO_LED_PIN)
    return cyw43_arch_init();
#endif
}

void pico_set_led(bool on) {
#ifdef PICO_DEFAULT_LED_PIN
    gpio_put(PICO_DEFAULT_LED_PIN, on);
#elif defined(CYW43_WL_GPIO_LED_PIN)
    cyw43_arch_gpio_put(CYW43_WL_GPIO_LED_PIN, on);
#endif
}

void servo_init(void) {
    gpio_set_function(SERVO_PIN, GPIO_FUNC_PWM);
    servo_slice = pwm_gpio_to_slice_num(SERVO_PIN);
    pwm_config config = pwm_get_default_config();
    pwm_config_set_clkdiv(&config, 125.0f);
    pwm_config_set_wrap(&config, 20000);
    pwm_init(servo_slice, &config, true);
    pwm_set_gpio_level(SERVO_PIN, SERVO_OFF_US);
}

void servo_set(bool on) {
    pwm_set_gpio_level(SERVO_PIN, on ? SERVO_ON_US : SERVO_OFF_US);
}

static void init_outputs(void) {
    gpio_init(EXT_LED_PIN);
    gpio_set_dir(EXT_LED_PIN, GPIO_OUT);
    gpio_put(EXT_LED_PIN, ext_led_state);
    servo_init();
    servo_set(servo_state);
}

void gpio_callback(uint gpio, uint32_t events) {
    if (!(events & GPIO_IRQ_EDGE_RISE)) return;
}
```

```
    switch (gpio) {
        case BTN_EXT_LED: ext_led_state = !ext_led_state;
gpio_put(EXT_LED_PIN, ext_led_state); break;
        case BTN_SERVO: servo_state = !servo_state;
servo_set(servo_state); break;
        case BTN_INTERNAL: internal_state = !internal_state;
break;
        case BTN_SLEEP: sleeping = true;
break;
    }
}

static void setup_buttons(void) {
    const uint8_t buttons[] = {BTN_EXT_LED, BTN_SERVO, BTN_INTERNAL,
BTN_SLEEP};
    for (size_t i = 0; i < sizeof(buttons) / sizeof(buttons[0]); i++) {
        gpio_init(buttons[i]);
        gpio_set_dir(buttons[i], GPIO_IN);
        gpio_pull_down(buttons[i]);
        gpio_set_irq_enabled_with_callback(buttons[i], GPIO_IRQ_EDGE_RISE,
true, &gpio_callback);
    }
}

static void enter_dormant(void) {
    sleep_run_from_xosc();
    sleep_goto_dormant_until_pin(BTN_SLEEP, false, false);
    sleep_power_up();
    init_outputs();
    setup_buttons();
    sleeping = false;
}

int main() {
    stdio_init_all();
    init_outputs();
    setup_buttons();
    while (true) {
        if (sleeping) { enter_dormant(); continue; }
        if (internal_state) {
            pico_set_led(true);
            sleep_ms(LED_DELAY_MS);
            pico_set_led(false);
        }
        sleep_ms(LED_DELAY_MS);
    }
}
```

3. Multicore

Q1. What cores does the RP2040 (Pico W) have and what cores does the RP2350 (Pico 2) have?

The RP2040 is the first-generation Raspberry Pi Pico (Pico 1, Pico W). The RP2350 is the Pico 2 generation (Pico 2, Pico 2W).

- **Pico 1:** Dual-core Arm Cortex-M0+ @ 133MHz
- **Pico 2:** Hybrid system — Arm Cortex-M33 and Hazard3 RISC-V cores. Both are dual-core and you can choose which ISA to use in the boot options.

Q2. What statements regarding the cores in the RP2040 are correct? (Multiple Choice)

- A. One of the cores is dedicated to the I/O
- **B. The cores are identical to each other ✓**
- C. One core controls the other
- **D. Each core is independent ✓**
- **E. One of the cores is asleep until activated ✓**

Q3. What do the cores on the RP2040 (Pico W) share? (Multiple Choice)

- **A. SRAM ✓**
- B. Interrupt hardware
- C. Program counter
- **D. Clock system ✓**
- **E. Bus fabric ✓**
- F. CPU registers
- G. Interrupt handling

Q4. Describe how the 2 cores in the RP2040 communicate with one another.

The two cores communicate primarily through the SIO (Single-cycle I/O) block using two hardware FIFO (First-In, First-Out) buffers.

- One FIFO is for Core 0 sending data to Core 1.
- The other FIFO is for Core 1 sending data to Core 0.
- Each FIFO is 32-bits wide and 8 entries deep.
- They can also use shared SRAM for larger data structures, using the FIFOs to pass memory pointers.

Q5. How does the shared memory of the cores operate? Explain also what risks could be involved.

- **Operation:** The 264KB SRAM is split into 4 main striped banks (plus 2 smaller scratchpad banks). The bus fabric allows both cores to access *different* memory banks simultaneously at full speed without stalling each other.
- **Risks:** The primary risk is a race condition or data corruption. If both cores try to read and write to the exact same memory variable at the same time, one core might read stale data, or one core might overwrite changes made by the other before they are finished.

Q6. What is the main difference between multicore on the RP2350 compared to the RP2040?

The RP2350 allows choosing between dual Arm Cortex-M33 or dual RISC-V cores. Additionally, the RP2350's M33 cores support Arm TrustZone, meaning multicore processing can be split into secure and non-secure environments — something the RP2040 cannot do. The RP2350 also upgrades the FIFOs to be deeper (16 entries instead of 8) and added a double-write feature.

Q7. What happens when 2 cores want to access the same SRAM?

- A. The system crashes
- **B. The access is arbitrated, causing delays ✓**
- C. One core gets disabled for X amount of time
- D. One of the cores gets redirected to another memory location automatically

Q8. How do hardware spinlocks work?

Hardware spinlocks are dedicated registers in the SIO block used to claim shared resources.

1. A core reads a specific spinlock register.
2. If the spinlock is free (0), the hardware automatically sets it to locked (1) and returns the “free” status to the core in a single, atomic operation.
3. If it was already locked, it returns a “locked” status. The core will then “spin” (loop and keep checking) until the other core releases it.

Q9. How is access to shared SRAM physically organized in the RP2040?

It is organized using bus striping. The 264KB memory is physically divided into 4 independent 64KB banks (Banks 0–3) and two 4KB scratchpad banks. Successive memory addresses point to alternating banks. Because each bank has its own hardware interface to the AHB-Lite bus fabric, Core 0 can access Bank 0 while Core 1 accesses Bank 1 at the exact same time with zero interference.

Q10. What is the role of the bus fabric in multicore and why is it so important?

The bus fabric acts like a high-speed traffic controller connecting the masters (Cores, DMA) to the slaves (SRAM, Peripherals). It allows parallel routing, preventing the system from bottlenecking by allowing multiple data transfers to happen simultaneously, managing conflicts smoothly via hardware arbitration when collisions occur.

Q11. What hardware mechanism ensures that two cores do not corrupt shared resources simultaneously?

- A. Cache coherence protocol
- **B. Hardware spinlocks ✓**
- C. Bus arbitration
- D. Memory mirroring

Q12. What is the role of the SIO (Single-cycle I/O) block in multicore operation?

The SIO is a dedicated hardware block that sits directly on the processor's local bus (bypassing the main bus fabric) so it can respond in exactly one clock cycle. In multicore operations, it houses the Inter-core FIFOs, the Hardware Spinlocks, and core-specific CPU IDs.

Q13. What causes a race condition?

- A. When two cores compete to run faster
- **B. When program execution depends on timing/order of access to shared data ✓**
- C. When memory runs out
- D. When threads are executed sequentially

Q14. How do the two cores in the RP2040 communicate? Describe the process.

1. **Core 0** writes a 32-bit value (like a command or a memory pointer) to its SIO FIFO output register.
2. Writing to this register automatically triggers an **interrupt (IRQ)** on **Core 1** (if enabled), or Core 1 can manually poll the FIFO status.
3. **Core 1** reads the value out of its SIO FIFO input register.
4. Once read, the slot in the 8-entry FIFO clears up, allowing Core 0 to send more data.

Q15. Give a task that benefits from multicore processing.

An excellent example is Audio/Video processing combined with User Interface handling. Core 0 can handle time-critical, heavy computational tasks like decoding an audio stream or reading a camera sensor, while Core 1 simultaneously handles rendering graphics on an LCD screen and polling buttons — without causing lag or audio stuttering.

Q16. Are both cores active from the start in the RP2040?

No. At boot, only Core 0 is active. Core 1 is held in a reset state inside a low-power wait loop (**WFE** — Wait For Event instruction). Core 0 must explicitly wake up Core 1 by using the SDK's `multicore_launch_core1()` function, which passes a boot vector through the SIO FIFO.

Q17. What happens when writing to a full FIFO buffer on the RP2350?

When a core attempts to write to a full FIFO on the RP2350, the processor stalls (blocks). The hardware pauses the core's execution on that instruction until the receiving core reads data out of the FIFO, freeing up a slot.

Q18. What happens when both cores try to acquire a spinlock at the same time in the RP2350?

The underlying hardware arbitration guarantees that one core will always hit the register a fraction of a cycle before the other, or the bus arbitrator will choose one. Only one core will successfully receive the “unlocked” status and claim it. The losing core will receive a “locked” status, ensuring complete mutual exclusion without any tie scenarios.

3.1. Assignment Part 2: Multicore Communication with Strings

3.1.1. Changes Made: Strings Instead of Chars

To use strings instead of characters, the following architectural changes were made:

1. **Memory sharing:** In the RP2040, both cores share the same memory space. Instead of pushing the actual data through the FIFO (which is only 32 bits wide), the **memory address** (pointer) of a string buffer is pushed instead.

2. Buffering in Core 0:

- A `message_buffer` (char array) and a `buffer_index` were added.
- Core 0 now collects characters from `stdio` and stores them in the buffer.
- When a newline (`\n` or `\r`) is detected, it null-terminates the string and sends the pointer to Core 1.

3. Core 1 processing:

- The IRQ handler on Core 1 pops the 32-bit value from the FIFO.
- It casts this value back to a `char *`.
- It uses `printf` with the `%s` format specifier to print the entire string.

3.1.2. Technical Note

Since both cores access the same `message_buffer`, there is a potential race condition if Core 0 starts writing a new message before Core 1 finishes printing the old one. In this implementation, the `printf` on Core 0 and the delay in the loop provide enough time for Core 1 to process the string, but in a production environment, a semaphore or mutex should be used to synchronize access to the shared buffer.

4. Security & OTP

4.1. Step 1: Generate a Private Key

The following command generates a private key in PEM format:

```
openssl ecparam -name prime256v1 -genkey -noout -out private_key.pem
```

BASH

4.2. Step 2: Updating CMakeLists

The assignment required signing the program. The `pico_sign_binary` function was added right after `target_link_libraries`:

```
pico_sign_binary(${PROJECT} KEY private_key.pem)
```

CMAKE

4.3. Step 3: Checking the Build Files

After running the build, a new file with the `.json` extension appeared in the build folder (e.g. `otp_config.json`). This file contains the hash of the public key, which is what actually gets programmed onto the chip.

4.4. Step 4: OTP Loading

The final step is loading the config onto the Pico 2. The command is:

```
picotool otp load trusted_program.json
```

BASH

Important: OTP stands for One-Time Programmable. Mistakes here can permanently brick the board — proceed with caution.

5. OTA Updates

Q1. What is the primary bootloader and what is its role in OTA?

The primary bootloader is read-only code burnt into the chip's ROM. It runs first at boot to perform basic hardware setup and then points the chip to the next-stage software, so an OTA update can safely start executing.

Q2. What happens if the update fails?

The system triggers a rollback mechanism where it marks the new firmware as invalid, reboots, and automatically falls back to the last known working version so the device is not bricked.

Q3. When is a second-stage bootloader necessary, and what makes it different from the primary one?

It is needed when using an RTOS or complex partition setups to switch firmware files. It differs because it lives in modifiable flash memory, whereas the primary bootloader is unchangeable ROM.

Q4. Why have OTA support via a bootloader rather than within the main program?

Doing it via a bootloader provides a clean, safe environment separate from the main application context, preventing critical crashes that can occur if you try to swap application files while an RTOS is actively running.

Q5. Does the entire OTA process need to be done by the bootloader?

No. The active user application normally does the heavy lifting — downloading the update over Wi-Fi and streaming it into flash — while the bootloader just takes over after a reset to verify and officially boot the new firmware.

Q6. How many types of resets for the ESP32 are there, and what are they?

There are three types shown in the workflow:

- Power-on reset (POR)
- Software reset
- Reset from deep sleep

Q7. In one sentence, what decision is the ESP32 making during secure OTA?

The ESP32 is deciding whether the downloaded firmware metadata has a valid cryptographic signature from a trusted source and matches the actual downloaded binary bytes before choosing to boot it.

Q8. In the successful “good” case, which monitor lines prove that the update was accepted for cryptographic reasons?

The lines showing `verify_payload_signature()` returning `ESP_OK` and the logs verifying that the calculated SHA-256 hash perfectly matches the expected value without any errors.

Q9. Why does `ota_demo_start_from_manifest_url()` verify the manifest before downloading and installing the firmware image?

Verifying the manifest first ensures the metadata is authentic, stopping the device from wasting time, network bandwidth, and flash wear by downloading a corrupt or malicious payload.

Q10. What does `http_get_to_buffer()` protect against with `MANIFEST_MAX_BYTES`?

It protects against buffer overflows and memory exhaustion by ensuring a malicious server cannot crash the device by sending a huge payload that exceeds the chip's small SRAM buffers.

Q11. Why is the image hash inside the signed manifest instead of being trusted from a separate unsigned file?

Putting the hash inside the signed manifest binds it directly to the digital signature, meaning an attacker cannot modify the binary and simply spoof a separate unsigned hash file without breaking the signature check.

Q12. Does Base64 make the manifest secure? Explain using this project.

No. Base64 is just text encoding for data transfer and provides zero security. The actual safety comes entirely from the asymmetric ECDSA cryptographic signature attached to the manifest.

Q13. The ESP32 prints `manifest download failed: ESP_ERR_HTTP_CONNECT`. Name two likely causes in this demo setup.

1. The local laptop hosting the manifest file is not running its web server, or a firewall is blocking the connection.
2. The ESP32 failed to connect to the local Wi-Fi network or has an incorrect target IP address configured.

Q14. In the “tamper” case, why can the manifest signature pass while the update is still rejected?

The manifest signature passes because the manifest file itself is unaltered and authentic, but the update is rejected because the actual downloaded firmware binary bytes were tampered with, creating a hash mismatch against the signed definition.

6. USB as Host

NRZI Decoding Exercise

Answer in hexadecimal: `00 07 F0`

Decoding process: USB uses NRZI encoding where a transition (J→K or K→J) represents a `0` and no transition represents a `1`. It also uses LSB-first bit ordering.

- Start state: J
- K (transition) → `0`
- J (transition) → `0`
- J (no transition) → `1`
- J (no transition) → `1`
- K (transition) → `0`
- K (no transition) → `1`
- K (no transition) → `1`
- K (no transition) → `1` → Byte 1 complete: `11101100` → LSB-first: `00110111` = `0x37`

The first byte is `00` due to the sync pattern; bytes 2 and 3 decode to `07` and `F0`.

Why does USB use differential signaling instead of a separate clock wire?

USB uses differential signaling on the D+ and D- wires to provide high noise immunity against external electromagnetic interference. Any noise affects both wires equally, so the receiver subtracts the difference to recover a clean signal — without needing an extra clock line.

For what purpose does USB use bit stuffing?

USB uses bit stuffing to force a signal transition after six consecutive `1` bits, so the receiving device can synchronize its internal clock with the incoming data stream and prevent timing drift.

Name one way USB maintains the integrity of data being sent.

USB maintains data integrity by using Cyclic Redundancy Checks (CRC) attached to data packets, allowing the receiver to detect if any bits were corrupted during transmission.

How does the host know the speed of a device?

The host determines device speed by detecting which data line has a pull-up resistor attached. A pull-up on D+ indicates a full-speed or high-speed device; a pull-up on D- indicates a low-speed device.

In TinyUSB, how do you configure your microcontroller to act as a host?

Define `CFG_TUSB_RHPORT0_MODE` as `OPT_MODE_HOST` in your `tusb_config.h` file, then call `tuh_init()` inside your main initialization code.

What is a S0F and why is it necessary?

A S0F (Start of Frame) is a special packet sent by the host every 1 millisecond (for full speed). It acts as a heartbeat to keep all connected devices synchronized and manages the timing of scheduled data transfers.

7. Bluetooth

7.1. Assignment BLE — Multiple Choice & Short Questions

Q1. Which company originally developed the foundation of BLE under the name “Wibree” in 2006?

- A. Apple
- **B. Nokia ✓**
- C. Infineon
- D. STMicroelectronics

Q2. The Raspberry Pi Pico W uses two main chips. Which chip handles the Wi-Fi and BLE wireless communication?

- A. RP2040
- B. STM32
- **C. CYW43439 ✓**
- D. ESP32

Q3. Through which internal hardware bus protocol do the RP2040 and the wireless co-processor communicate on the Pico W?

- A. I2C
- B. UART
- C. USB
- **D. SPI ✓**

Q4. How many RF channels are used by Bluetooth Low Energy (BLE) compared to the 79 channels used by Bluetooth Classic?

- A. 20
- **B. 40 ✓**
- C. 50
- D. 100

Q5. Which protocol layer is primarily responsible for handling network discovery, broadcasting advertisement packets, and managing connection roles?

- A. GATT (Generic Attribute Profile)
- **B. GAP (Generic Access Profile) ✓**
- C. Physical Layer (PHY)
- D. L2CAP

Q6. In the GATT data hierarchy, what is the term used for a collection of characteristics that represent a specific function (like the Environmental Sensing Service)?

- A. Profile
- B. Descriptor
- **C. Service ✓**
- D. UUID

Q7. Bluetooth SIG standardized 16-bit UUIDs are used for common data types. What is the standard 16-bit UUID assigned to the Temperature characteristic?

- A. 0x180F

- B. 0x181A
- C. 0x2A37
- D. 0x2A6E ✓

Q8. Which BTstack library function is used in code to actively push updated sensor measurements from the GATT server to a connected smartphone client?

- A. `cyw43_arch_init()`
- B. `att_server_notify()` ✓
- C. `l2cap_init()`
- D. `hci_power_control()`

Q9. Why does a Bluetooth Low Energy device typically have a battery life that lasts for months or years compared to a Bluetooth Classic device?

BLE was designed around the idea of sending tiny data bursts and then switching the radio completely off between transfers. Classic Bluetooth holds a constant connection open the whole time, which keeps the radio active and drains the battery much faster. That's why you can run a BLE temperature sensor off a coin cell for a year while a Classic BT device needs regular charging.

Q10. When a smartphone connects to your Pico W to check data, which device acts as the GATT Server, and which device acts as the GATT Client?

The Pico W is the **GATT Server** — it's the one that owns and exposes the sensor data. The phone is the **GATT Client**, connecting to the Pico and requesting those readings.

7.2. Assignment: Classic Bluetooth SPP Server

The task was to fill in the missing packet handler logic in `classic_bluetooth-2/main.c` and register a custom device name so it would appear correctly when pairing from a phone or PC.

7.2.1. What I Implemented

1. **Device name:** Registered the board as `"nw1728-bt"` via `gap_set_local_name()` so it shows up with a recognizable name during pairing.
2. **SSP pairing:** Inside `HCI_EVENT_USER_CONFIRMATION_REQUEST`, extracted the 6-digit numeric confirmation code with `little_endian_read_32(packet, 8)` and accepted it using `gap_ssp_confirmation_response()`.
3. **Accepting incoming connections:** In `RFCOMM_EVENT_INCOMING_CONNECTION`, printed the connecting device's address and called `rfcomm_accept_connection()` to let it through.
4. **Channel open and close:** On `RFCOMM_EVENT_CHANNEL_OPENED`, checked the status and stored the MTU. On `RFCOMM_EVENT_CHANNEL_CLOSED`, cleared the stored channel ID so the handler knows the link is gone.
5. **Receiving data:** In the `RFCOMM_DATA_PACKET` case, printed whatever bytes arrived over the serial link to stdout.

8. DMA

8.1. Assignment: DMA Demo

Q1. What is the purpose of each of the following variables?

```
const char src[] = "This text was copied by DMA!";  
char dst[sizeof(src)] = {0};  
  
int dma_chan;  
volatile bool dma_done = false;
```

- `src[]`: The source data in memory that DMA will read from.
- `dst[]`: The destination buffer in SRAM where the copied bytes land. Zero-initialized to avoid leftover garbage.
- `dma_chan`: Stores the channel number the SDK gave us when we claimed a DMA channel.
- `dma_done`: Declared `volatile` so the compiler doesn't optimize away reads of it. The ISR flips it to `true` when the transfer finishes, signaling the main loop it can continue.

Q2. What does DMA stand for?

Direct Memory Access.

Q3. What is the main advantage of DMA compared to CPU-based data transfers?

Without DMA, the CPU has to sit in a loop moving one byte at a time, blocking everything else. With DMA, the hardware handles the copy while the CPU is free to run other code, sleep, or handle interrupts. This is a big deal for things like audio streaming or peripheral data logging where you're constantly moving large chunks of data.

Q4. How many DMA channels are available on the RP2040?

12 channels, all independently configurable.

Q5. What are the four main DMA channel registers on the RP2040?

1. `READ_ADDR` — the address DMA reads from
2. `WRITE_ADDR` — the address DMA writes to
3. `TRANS_COUNT` — how many elements to transfer
4. `CTRL_TRIG` — channel configuration and the trigger that starts the transfer

Q6. What is the purpose of the `TRANS_COUNT` register?

It tells the DMA controller how many data elements to move before stopping. Once it hits zero, the channel halts and can optionally fire an interrupt or chain to another channel.

Q7. What is a DREQ signal and why is it used?

DREQ (Data Request) is a pacing signal from a peripheral that tells DMA when it is actually ready to accept or provide data. Without it, DMA would push data faster

than the peripheral can keep up with, causing overflows or underflows. DREQ keeps DMA in sync with the peripheral's real throughput.

Q8. What is DMA chaining?

When one DMA channel finishes, it can automatically kick off another channel. This is useful for double-buffering or multi-stage pipelines where you need a continuous stream of transfers without the CPU having to restart each one manually.

Q9. What are DMA aliases and why does the RP2040 provide multiple aliases for each DMA channel?

Each channel's registers are mapped at four different base addresses. They all point to the same underlying hardware, but writing to the last register in each alias triggers the channel — and each alias puts a different register in that trigger slot. This lets you update just the source address, or just the count, and fire the channel without touching all the other fields.

Q10. Describe the general DMA workflow from configuration to completion.

1. **Claim:** Request a free channel from the SDK with `dma_claim_unused_channel()`.
2. **Configure:** Fill a `dma_channel_config` struct — set transfer width, address increment flags, and the DREQ source.
3. **Trigger:** Write the source/destination addresses and element count, then write `CTRL_TRIG` to start the transfer.
4. **Complete:** DMA finishes in the background and fires an IRQ (or chains). The ISR sets a flag that the main code checks before touching the transferred data.